VITALS SCREENING APP • SEVEN DAY BUILD PLAN

A phone they already carry.
Turned into a life saving screening tool.

Vytal reads a person’s heart rate, breathing rate and stress level from a ten second camera scan, explains the reading out loud in plain language, and makes sure that a flagged referral never quietly gets lost. This plan lays out what gets built, by whom, for each of the next seven days, and what the finished app looks like at the end of it.

TEAM VYTAL

Ahmad, AI, APIs and Backend
Muhammad Ahmad, Frontend and UI
Laiba, Backend

WHY WE ARE BUILDING THIS

One missing tool. One broken promise.

In a lot of rural clinics, refugee camps and low income communities, a community health worker is often the first, and sometimes the only, person who checks on someone who is unwell. The problem is these workers usually do not carry a pulse oximeter or a blood pressure cuff. So instead of a real number, care depends on how a person looks and how they say they feel.

Then a second problem sits right behind the first one. When a worker does notice something serious and tells the person to go to a real clinic, a surprising number of those people never actually get there. Studies from rural Uganda, Zambia and Haiti all found the same pattern. The referral gets made, someone says go see a doctor, and then it quietly falls apart. No phone call, no record, no one following up. By the time anyone notices, it is too late to matter.

So the real gap is not one problem, it is two, stacked on top of each other. No easy way to get an honest reading, and no reliable way to make sure a flagged concern actually gets followed through.

THE IDEA

The phone already in their pocket, becomes the missing piece.

Someone looks into the camera for about ten seconds. From that short scan alone, Vytal works out three things: heart rate, breathing rate and a simple stress indicator. Once it has a reading, Vytal hands the numbers to an AI model named Qwen, which turns something like heart rate ninety two, stress a little high, into a calm, plain explanation in whatever language the patient actually speaks, and gently flags if a real doctor should take a look.

If a referral gets flagged, it does not just get said out loud and forgotten. It gets attached to that person's record right there on the spot, and Vytal keeps working with zero internet connection, quietly syncing everything the moment a signal comes back.

> Camera based heart rate apps already exist, they just show a number and stop there. Health worker platforms like CommCare already exist too, they are excellent at tracking patients but cannot take a reading. Vytal sits in the gap between the two, it is the sensor those platforms do not have, and the follow through that heart rate apps never bothered to build.

THE CORE IDEA, EXPLAINED SIMPLY

A camera can feel a heartbeat.

Here is the part that sounds like magic but is actually simple physics. Every single heartbeat pushes a small pulse of blood through the skin. That tiny pulse changes how much light the skin reflects for a moment, in a way no human eye can notice, but a camera can. Point a phone camera at a face or a fingertip for about ten seconds, and that tiny flicker shows up as a small colour change across many video frames.

Skin → Tiny colour change with every heartbeat → Camera records it, frame by frame → Software removes shaky hands and bad light → Software counts the beats per minute

If the room is too dark, or the camera is not good enough for a clean face reading, Vytal switches modes on its own. It asks the person to place a finger over the phone's rear camera and flash instead. The light source is now fixed instead of bouncing around a room, and it works even on old, cheap phones.

> This single idea is the seed for everything else in the app. The reading it produces becomes the sentence Ahmad's AI layer explains, the record Laiba and Ahmad's backend saves and syncs, and the number Muhammad Ahmad's dashboard shows the health worker.

WHO BUILDS WHAT

Three people.
Three lanes. One app.

Ahmad. AI, APIs and Backend.

- Owns the Qwen AI layer, turning a raw reading into a calm, plain sentence
- Owns every outside connection, Alibaba Cloud, Model Studio and the referral flag logic
- Shares the backend with Laiba, since the same person wiring the API also wires what stores its output
- Picks up any task that does not sit cleanly in the other two lanes

Muhammad Ahmad. Frontend and UI.

- Owns the camera capture screen, the face scan and the fingertip and flash fallback
- Owns the health worker dashboard and the printable one page report
- Owns every visual detail a patient or health worker actually sees and taps
- Keeps all three screens feeling like one connected app, not three separate ones

Laiba. Backend.

- Owns the offline first storage, so a reading is never lost with no signal
- Owns the sync logic that quietly uploads everything once internet returns
- Works side by side with Ahmad on the cloud database and the deployment
- Keeps patient data organised and easy to find again later

Whenever a task does not fit neatly into one of these three lanes, it becomes Ahmad's task too, since Ahmad already understands both the API layer and the backend underneath it.

DAY ONE

Foundations.

Goal for today: get every account, tool and empty screen ready, so real building can start tomorrow with zero setup delay.

Ahmad

- Create the Alibaba Cloud account and turn on Model Studio
- Get a working DashScope API key, send one test message to Qwen and confirm it replies
- Write down the exact shape of a reading record, heart rate, breathing rate, stress score, timestamp, patient id, referral flag

Muhammad Ahmad

- Set up the shared repository and three empty screens, scan screen, dashboard, report
- Bring the already built face scan code into the shared repository
- Sketch the look of the dashboard and report before building either

Laiba

- Create the Alibaba Cloud storage space that will eventually hold synced readings
- Read through the offline queue plan with Ahmad, agree on one shared data shape
- List every piece of information a synced record needs to carry

DAY TWO

The first real signal.

Goal for today: get a real heart rate number on screen from an actual camera, and a real sentence back from Qwen.

Ahmad

- Wire a real reading into a Qwen prompt and get back a plain language explanation
- Add the referral rule, a simple test for when a reading should suggest seeing a clinician
- Test the same call through Groq as a backup path for later

Muhammad Ahmad

- Get the face scan producing a live heart rate number end to end, on screen
- Add a simple live signal quality readout, so the scan shows if it is working
- Test the scan in a normal room and confirm the number looks reasonable

Laiba

- Start the offline first storage using IndexedDB, so a reading is never lost
- Build the basic save function, a reading is stored the instant it is captured
- Confirm with Ahmad that the local record and the cloud record match

DAY THREE

The fallback, and the language.

Goal for today: make sure a bad camera or a dark room never means no reading, and every explanation returns in the right language.

Ahmad

- Add a language selector, so Qwen replies in the patient's own language
- Refine the prompt so explanations stay calm, short and free of scary words
- Begin the Function Compute endpoint that will receive synced readings

Muhammad Ahmad

- Build the fingertip and flash capture mode for when the face scan is too weak
- Build the automatic switch that moves to fingertip mode on its own
- Test both modes once in a dim room and once in a bright room

Laiba

- Build the sync function that uploads local readings once internet returns
- Add a visible offline indicator, so it is always clear what has synced
- Connect the sync function to Ahmad's new Function Compute endpoint

DAY FOUR

The dashboard comes alive.

Goal for today: give the health worker one clear screen that lists every patient, and shows exactly who needs a second look.

Ahmad

- Finish the referral flag logic, so it reliably marks a reading for follow up
- Add basic error handling, so a slow AI reply never freezes the app
- Structure the cloud backend so flagged referrals are easy to find later

Muhammad Ahmad

- Build the dashboard, patient list, last reading, referral flag, one screen
- Make the referral flag impossible to miss, using colour and placement
- Connect the dashboard to real local data instead of placeholder numbers

Laiba

- Finish the cloud database structure so queries stay fast with many patients
- Add basic security, so patient data cannot be read from outside the app
- Test that a reading saved offline still appears correctly once it syncs

DAY FIVE

The report, and the proof.

Goal for today: give every patient a one page, printable record of what happened, and prove the cloud side genuinely works.

Ahmad

- Finish the Alibaba Cloud deployment, capture a screenshot or clip as proof
- Polish the explanation text on the printed report, a patient may read it alone
- Run ten sample readings, check nothing sounds confusing or alarming

Muhammad Ahmad

- Build the one page report, reading summary, explanation, QR code to the record
- Make sure the report still looks clean once actually printed on paper
- Polish the visual style so all three screens feel like one connected app

Laiba

- Make sure every synced reading can be found again by patient id
- Run a full offline test, three readings with no internet, then reconnect
- Save a copy of the cloud setup steps in case anything needs rebuilding

DAY SIX

Everything, together.

Goal for today, all three together: connect every piece into one single flow, and break it on purpose so it gets fixed before anyone else sees it. Walk through scan, explanation, flag, dashboard and report at least five times, fix whatever breaks in whatever lane it lives in, and agree on one synthetic demo mode as a safety net.

Ahmad watches for

- Any moment where a slow AI reply makes the whole flow feel stuck
- A small loading message added wherever that happens

Muhammad Ahmad watches for

- Any screen that looks unfinished sitting next to the others
- Matching spacing, colour and font size across all three screens

Laiba watches for

- Any reading that fails to sync silently, without anyone noticing
- Turning silent failures into something visible on screen instead

DAY SEVEN

Polish, and the story.

Goal for today: make the app feel finished, and make sure everyone in the room can explain why it matters in one sentence.

Ahmad

- Read every AI explanation out loud, as if speaking to a real patient
- Confirm the Alibaba Cloud proof is saved somewhere safe for the pitch
- Prepare a short answer for, why Qwen and not just any AI model

Muhammad Ahmad

- Final visual pass, spacing, colour, font size, nothing left placeholder
- Rehearse the live demo three times, including the fingertip fallback
- Prepare a backup demo video in case live camera conditions are poor

Laiba

- Confirm offline mode truly works with the internet fully off, not just slow
- Double check patient data is not exposed anywhere it should not be
- Prepare a one line answer for, what happens if the cloud goes down

SEVEN DAYS FROM NOW

One phone.
Four screens. Zero gaps.

A health worker opens the app to a single button that starts a check. They point the camera at a person's face for about ten seconds, or switch to the fingertip and flash mode if the room is dark. A reading appears, heart rate, breathing rate and a simple stress read out. Below it sits a calm sentence explaining what the numbers mean, written in the patient's own language, with a clear note if a real clinician should take a look.

That reading, and the flag if there is one, saves to the phone immediately and quietly climbs to the cloud the moment a signal appears, with nothing lost in between. The dashboard lists every patient a health worker has checked, their last reading, and who is waiting on a follow up, all in one glance. Each patient also gets a one page printable report with a QR code, so a paper trail exists even where printers and signal both come and go. None of it needs new hardware. It all runs on the phone already in a health worker's pocket.

Camera reading → AI explains it → Saved on the phone → Synced to the cloud → Shown on the dashboard → Printed as a report

Not a medical device. Vytal explains and flags. The real decision always stays with a real clinician.

LET’S BUILD IT

Ready to build, day by day.

Seven days, three people, one reading that starts every screen after it. That is the whole plan.

TEAM VYTAL

Ahmad, AI, APIs and Backend
Muhammad Ahmad, Frontend and UI
Laiba, Backend

Vytal. The sensor community health platforms never had.